

Pokémon TCG Legacy — Configuration Guide

A living reference for every JSON-driven knob in the game. Each section lists the **field name**, the **allowed values**, the **file(s) that consume it**, and a **minimal example**.

Keep this file open while editing data files in <, <, <, and <.

1. Battle Conditions (Deck Restrictions)

Opponents can demand the player meet specific deck requirements before a battle starts. The restriction is attached to an opponent entry inside any < file under the < array.

Validator: < **Popup UI:** < **Trigger:** < runs validation just before the battle scene loads. If validation fails, a popup (or text-only message) is shown and the battle is aborted.

1.1 Display modes

Mode	Triggered by	What the player sees
<	< / < hit	Full-screen grid titled " Banned cards " showing every card on the NPC's banlist
<	Structural rule violated (type, set, supertype, HP, etc.)	Full-screen grid titled " Cards in your deck that aren't allowed " showing the offending cards from the player's deck
<	Required-card quantity not met	Standard messagebox text only, e.g. <

If multiple categories fail at once, priority is **banned** > **invalid** > **missing**. Missing-requirement lines are appended to the popup body when an <-mode popup is shown so the player sees everything at once.

1.3 Field reference

Field	Type	Notes
<	<	Exact card ids (<). Case-insensitive.
<	<	Card names. Matches across all sets, so < catches every reprint.
<	<	Per-card minimum quantity; matches one exact id.
<	<	Per-name minimum quantity; sums copies across all sets. Use this for energies (e.g. <) because energy cards reprint into many sets.
<	<	Total Stage-2 Pokémon copies required.
<	<	Pokémon < field must intersect. Values: <, <, <, <, <, <, <, <, <, <.
<	<	Matched off the energy card's name (< → <). Different sets share the same name, so cross-set Fire Energies all pass.
<	<	Card id prefix (e.g. <, <, <). Applies to all card types.
<	<	Case-insensitive name prefix list. Use < (with trailing space) for Team-Rocket Dark Pokémon, < for Neo Destiny Light Pokémon.
<	<	Reverse of the above.
<	<	Forbid all Trainer cards.
<	<	Forbid any energy with subtype <.
<	<	Forbid any energy with subtype <.
<	<	Only < or < subtype Pokémon.
<	<	Pokémon with HP > N are invalid. Trainer/Energy cards ignore this.
<	<	Pokémon name must contain " δ" (the delta-species marker).
<	<	Any card whose < array contains < is invalid.

1.4 Example: gym challenge

```
{
  "name": "Gym Leader Erika",
  "meet_text": "As part of the Gym Challenge you may only use Grass Pokémon. No Trainers either!",
  // ... other opponent fields ...
  "restrictions": {
    "allowed_pokemon_types": ["Grass"],
    "no_trainers": true
  }
}
```

1.5 Set id reference (for <)

Set id	Set name
<	Base Set
<	Jungle
<	Fossil
<	Team Rocket
<	Wizards Promo
<	Gym Heroes
<	Gym Challenge
<--<	Neo Genesis / Discovery / Revelation / Destiny
<--<	Expedition / Aquapolis / Skyridge
<--<	EX era
<	Nintendo Promo
<--<	POP series
<	Southern Islands

2. Match Effects (Special Battle Rules)

Opponents can carry **match-wide rule modifiers** that are active for the whole battle — e.g. "all Fire Pokémon deal +20 damage", "no retreating", "every coin flip lands on heads". They are the in-battle counterpart to §1's pre-battle deck restrictions, and both blocks can sit on the same opponent entry.

Parser / rules API: < (the schema is also documented in its header comment) **Loaded by:** < → re-read in the match by < **Announced:** at the start of **each game** (after bench setup, before the opening coin flip) as a "SPECIAL MATCH RULES ARE IN EFFECT!" message sequence — one line per effect.

Works automatically with < (games 2 and 3 reload the scene and re-read the JSON) and alongside <.

2.1 Syntax

< is an **array of effect objects**, sibling of < on the opponent entry. Multiple effects stack in one battle.

```

{
  "name": "Gym Leader Blaine",
  // ... other opponent fields ...
  "restrictions": { "allowed_pokemon_types": ["Fire"] }, // optional, §1
  "match_effects": [
    { "type": "type_damage_bonus", "pokemon_type": "Fire", "amount": 20 },
    { "type": "no_retreat", "applies_to": "player" },
    { "type": "coin_flip_override", "result": "heads" }
  ]
}

```

Every effect object has:

Field	Required	Notes
<	yes	One of the effect types in §2.2. Unknown types print a warning and are ignored.
<	no	< (default) < <. < = the human, < = the CPU. For offensive effects it selects whose attacks/actions are affected; for defensive effects (<, <, <, <, <, <) it selects whose Pokémon are affected.
(params)	varies	Per-type parameters listed below.

Stacking rules: additive effects (damage bonuses, reductions, heals, modifiers) stack if listed twice. Override-style effects (<, <, <, <, <, <) — the **last matching entry in the array wins**.

2.2 Effect type reference

Damage rules

<	Params	What it does
<	<, <	Pokémon of that type deal +X attack damage (e.g. all Fire +20). Stacks.
<	<	All evolved (non-Basic) Pokémon deal +X attack damage.
<	<, <, < (each optional)	Per-stage flat attack bonus, e.g. Stage 1 +10, Stage 2 +20.
<	<, <	Damage dealt to Pokémon of that type is reduced by X (e.g. Grass take 20 less).
<	—	Weakness is skipped in damage calculation.
<	—	Resistance is skipped in damage calculation.
<	—	Both skipped (same as listing the two above).
<	—	All raw attack damage becomes 0. Attack effects still happen. Applied last — wins over every bonus.
<	—	Attacks lose their effects and only do printed damage. Variable attacks are flattened with no coin flips : a < attack just does 20, a < attack does 10.

Damage rules only affect **attack** damage. Self-damage, recoil, bench splash and poison ticks are untouched (poison has its own effect below).

Turn flow & resources

<	Params	What it does
<	<	Draw N cards instead of 1 at the start of each turn.
<	<	Opening hands are N cards instead of 7 (mulligan rules unchanged).
<	<	Players may attach N energy cards per turn instead of 1.
<	<	To play a Trainer you must first discard N other cards from your hand (CPU pays too, and demands more value before playing trainers).
<	<	Maximum bench size (combines with Narrow Gym — the lower cap wins).
<	—	Every knockout awards 2 prize cards instead of 1 (bench tokens still award none).

Retreat rules

<	Params	What it does
<	—	Retreating is blocked entirely. Beats < if both present.
<	—	Retreating costs no energy.
<	< (can be negative)	Added to every retreat cost, floored at 0 (e.g. +1 = everything pays one more).

Healing rules

<	Params	What it does
<	—	All healing is blocked — potions, powers, berries, attack heals, and the attach/evolve heal effects below. Beats every other healing effect. Berries are not consumed while blocked.
<	<	All healing is scaled (2 = doubled).
<	<	Every Pokémon in play (active + bench, both sides unless <) heals X HP between turns.
<	—	Attaching an energy card from hand fully heals that Pokémon.
<	—	Attaching an energy card from hand halves that Pokémon's current HP (rounded up to the nearest 10, never below 10 — 50 → 30).
<	—	Evolving fully heals the Pokémon.

Damage-counter **transfer** powers (Damage Swap etc.) are not "healing" and ignore these rules.

Status & powers

<	Params	What it does
<	—	Poison / Toxic / Burn / Sleep / Confusion / Paralysis cannot be applied.
<	<	The between-turns poison tick is scaled (2 = doubled).
<	—	All Pokémon Powers and Poké-Bodies stop working (rides the same gate as Muk's Toxic Gas). < is ignored — always both sides.

Coins, energy & HP

<	Params	What it does
<	<: < <	Every coin flip is forced to that result (animation still plays). Disables Sabrina's ESP re-flip.
<	—	Every attached energy counts as any type. Double Colorless still provides 2 units.
<	< (can be negative)	Every Pokémon's max HP is shifted by $\pm X$ for the whole game, floored at 10. Cards start play at the shifted max.

2.3 Examples

```
// Symmetric pair of simple rules
"match_effects": [
  { "type": "draw_count", "count": 2 },
  { "type": "double_prizes" }
]

// One-sided handicap battle: only the PLAYER cannot retreat,
// and only the OPPONENT's Pokémon heal at end of turn
"match_effects": [
  { "type": "no_retreat",      "applies_to": "player" },
  { "type": "end_of_turn_heal", "amount": 10, "applies_to": "opponent" }
]

// Full gauntlet: best-of-3 + deck restriction + several effects at once
{
  "name": "Arena Master",
  "match_format": "best_of_3",
  "restrictions": { "banned_card_names": ["Bill"] },
  "match_effects": [
    { "type": "type_damage_bonus", "pokemon_type": "Fire", "amount": 10 },
    { "type": "no_retreat", "applies_to": "player" },
    { "type": "coin_flip_override", "result": "heads" }
  ]
}
```

2.4 Testing day

< is a dedicated test day with **one opponent per effect** (33 total) lined up in Celeste Harbour, plus a best-of-3 combo opponent. Reach it with the < debug key (sets date 0). They all use the < deck and announce their rule as their greeting text.

2.5 Known limitations

- CPU attack **scoring** ignores attacker-side damage bonuses and

< — the AI still picks attacks normally under those rules.

- < conflicts with the two cards that manage their own HP

override: Koga's Ditto (Giant Growth) and Sabrina's Gastly (Gaseous Form).

- Under <, the cost is paid even if Mind Games or Chaos

Gym subsequently wastes the trainer.

3. NPC & Opponent Spawning Conditions

Any NPC or opponent entry may carry a `<` block. The entry only spawns if the condition evaluates to **true**.

Evaluator: `<` (MapManager.gd $\approx$ line 311) **Special case:** During the immediate return-to-map after a battle, a just-beaten opponent stays spawned long enough to play the result text, even if their `<` condition would normally hide them now.

3.1 Condition types

<code><</code>	Extra fields	Returns true when
<code><</code>	<code><</code>	The named opponent has been beaten at least once
<code><</code>	<code><</code>	The named opponent has NOT been beaten
<code><</code>	<code><</code>	Every name in the list has been beaten
<code><</code>	<code><</code>	At least one name in the list has NOT been beaten
<code><</code>	<code><</code>	At least one name in the list has been beaten
<code><</code>	<code><</code>	The named NPC has been talked to at least once
<code><</code>	<code><</code>	The named NPC has NOT been talked to
<code><</code>	<code><</code>	<code><</code> is truthy
<code><</code>	<code><</code>	<code><</code> is falsy or missing

3.2 Examples

```
// Spawn only after Misty has been beaten
"condition": { "type": "opponent_defeated", "target": "Gym Leader Misty" }

// Spawn until the player completes ALL six Pikachu Fans
"condition": {
  "type": "not_all_opponents_defeated",
  "targets": [
    "Pikachu Fan Marina", "Pikachu Fan Skye",
    "Pikachu Fan Cami",   "Pikachu Fan Juniper",
    "Pikachu Fan Raye",   "Pikachu Fan Leaf"
  ]
}

// Spawn after a custom progress flag has been set elsewhere
"condition": { "type": "flag_set", "flag": "got_starter_deck" }
```

4. Movement Patterns

< controls how a spawned NPC or opponent moves around the map. The default is <.

Implementation:

- <
- <
- <

4.1 Pattern values

Pattern	Behaviour	Extra fields
<	Stands still, randomly turns to face a new direction every 1–4 s	—
<	Stands still but cycles the < animation (used for swimmers)	—
< / < / < / <	Stationary facing the given direction; restored 1 s after a conversation	—
<	Walks back and forth along one axis	< (default <), < (default <), < (< or <)
<	Walks a square loop (down → right → up → left → repeat)	<, <
<	Picks short random walks within a radius of the spawn point	<, < (default <)

4.2 Extra-field reference

Field	Type	Default	Used by
<	float	<	<, <, <
<	float	<	<, <
<	< <	<	<
<	float	<	<

4.3 Examples

```
{ "name": "Biker2", "pattern": "patrol_line", "patrol_speed": 120,
  "patrol_distance": 210, "patrol_axis": "horizontal",
  "position": { "x": 400, "y": 1050 } }

{ "name": "Pikachu Fan Marina", "pattern": "random_wander",
  "patrol_speed": 20, "wander_radius": 65,
  "position": { "x": 1425, "y": 1000 } }

{ "name": "Fisherman Dave", "pattern": "idle_down",
  "position": { "x": -600, "y": 2730 } }
```

5. NPC Types

< decides which behaviour script handles the NPC and which scene is instantiated. Set on the NPC's entry in < (or on the per-map placement if it varies by location).

Dispatcher: < (≈ line 280) and < (≈ line 553).

<	Behaviour	Required extra fields
(omitted) / <	Plain NPC — shows < then <. Optionally a gift NPC if < is set.	—
<	Opens the juice-bar coin lottery	— (handled inline in MapManager)
<	Opens the corresponding shop scene; uses Shopkeeper state machine	<

5.1 Shop ids

<	Scene loaded	Notes
<	<	Standard pack shop; uses Day-1 starter-set state machine
<	<	Forces <
<	<	Reads <
<	<	—

5.2 Example shop NPC

```
"Holo Rare Shopkeeper": {  
  "sprite": "NPC_Holo_Rare_Shopkeeper",  
  "npc_type": "shop",  
  "shop_id": "holo_mart"  
}
```

6. Gifts (NPC-given)

A non-shop NPC becomes a **gift NPC** as soon as < is non-empty. The gift is awarded the first time the player talks to them, then the NPC falls back to its < from then on.

Handler: < (line 619) and < (line 666). **Detection:** <.

6.1 Gift types

<	< format	What it does	Visual reveal
<	Comma-separated card ids, e.g. <	Adds cards to the player's collection via <	Card-flip animation
<	Decimal string, e.g. <	<	Silent
<	Coin filename without <, e.g. <	<	Coin-flip animation
<	Costume key (case variants tolerated), e.g. <, <	<	Fade-in costume image
<	Style name, e.g. <	Appends to <	Silent
<	Single pack id, e.g. <	Appends to < — unlocks it for purchase in Card Mart	Silent
<	Comma-separated pack art codes, e.g. <	Queues < on the next OK press — opens packs immediately	Pack opening sequence
<	(Pack name)	Not implemented — currently emits a <	n/a

6.2 Special placeholders seen in data

Placeholder	Meaning
< in <	Used in early/draft data to mark "fill in a real coin later". Treat as a TODO marker.
< in < field	Same idea for opponent BGM.

6.3 Examples

```
// Card gift (single)
"Kid Opening Packs": { "sprite": "NPC 02", "gift_type": "card", "gift_value": "base1-20" }

// Multi-card gift
"Card Elder Hamish Day 4": { "sprite": "Elder", "gift_type": "card",
  "gift_value": "base1-1, base1-2, base1-3" }

// Cash
"Man Staring Out To Sea": { "sprite": "Youngcouple2_2",
  "gift_type": "cash", "gift_value": "250" }

// Costume
"Sailor Working On Dock": { "sprite": "Sailor",
  "gift_type": "costume", "gift_value": "Sailor" }

// Unlock a pack for sale in the Card Mart
"Card Mart Rep": { "sprite": "NPC 09",
  "gift_type": "available_pack", "gift_value": "base4" }

// Immediately open one or more packs
"Rocket Pack Gift": { "sprite": "Teamrocket_M",
  "gift_type": "pack", "gift_value": "base5_a, base5_b" }
```

6.4 Costume-gated NPCs

An NPC carrying a < field reacts to **what the player is currently wearing** (the < field of <).

Handler: <.

Field	Notes
<	Costume sprite name the player must be wearing. Compared case-insensitively. Presence of this field marks the NPC as costume-gated.
<	Greeting shown while wearing the required costume.
< / <	Greeting shown when not wearing it (ordinary text NPC behaviour).
< / <	Standard gift fields (§6.1). The gift is handed over only while the required costume is worn, and only once.

While wearing the costume the player gets < and the gift; otherwise they get the normal greeting and nothing — they can return later in the right outfit. Works with every < in §6.1.

```
// Gives a pack of cards, but only to a player dressed as a Rocket grunt
{
  "name": "Rocket Grunt", "sprite": "Teamrocket_M",
  "required_costume": "Teamrocket_M",
  "meet_text": "What are you looking at, kid? Beat it.",
  "repeat_text": "I told you to get lost.",
  "costume_match_text": "Hey, you're one of us! Here, take this pack.",
  "gift_type": "pack", "gift_value": "base1_a"
}
```

7. Opponent Rewards

Rewards are read after a winning match by < (around lines 283–327). They live on the opponent's entry in < (or on the per-map placement to override).

Field	Format	Awarded	Notes
<	Integer string, e.g. <	Every win	First win is tripled (×3) and the outro label shows <
<	Single coin filename without <, e.g. <	First win only , and only if the player doesn't already own that coin	Triggers coin-flip animation
<	Comma-separated card ids, e.g. <	First win only	Added via <
<	Comma-separated costume keys, e.g. <	First win only	Per key, only added if not already owned
<	Single pack art code, e.g. <	Not currently consumed — present on < but no script reads it. Treat as planned/not-wired.	
<	int (1–6)	—	Number of prize cards in the match itself (not a reward)
<	string (set id / <)	—	BGM track played during the battle
<	Deck name (string)	—	Looks up <

Are packs allowed as a prize? Not via < (currently unused).

If you want a battle to award a pack, the workable mechanism is to award a **card** via < and trigger pack-opening via a gift NPC elsewhere

with <. The < field is reserved for future wiring.

7.1 Example opponent

```
"Card Expert Nathan": {
  "sprite":      "Cooltrainer_M3",
  "deck":        "Energy Burn",
  "music":       "REPLACEMUSIC",
  "prize_cards": 6,
  "cash_reward": "150",
  "coin_reward": "Charizard Red",
  "card_reward": "basep-3, basep-14",
  "costume_reward": "Cooltrainer_M3"
}
```

8. Map Files: Day & Time Variants

Each overworld map loads a different JSON file depending on the current **date** and **time of day**.

Loader: < (≈ line 16) **State source:** < and <.

8.1 Filename convention

<

Part	Allowed values
<	<, <, ... (one per overworld scene)
<	Integer day number — <, <, <, <, <, ...
<	<, <, <

So:

```
NPC_and_Opponent_Data/Celeste_Harbour_1_Day.json
NPC_and_Opponent_Data/Celeste_Harbour_3_Night.json
NPC_and_Opponent_Data/Verdant_Forest_4_Evening.json
```


8.2 File top-level shape

```
{
  "npcs":      [ { ...npc placement... } ],
  "opponents": [ { ...opponent placement... } ]
}
```

Both arrays may carry per-entry `<` blocks (see §3) to toggle entries on or off based on progress.

8.3 Time cycle

Time advances **Day** → **Evening** → **Night** → **Day** (Day rollover bumps `<`). Day-driven map events (e.g. boats appearing, beach blocked, SS Anne docking) are scripted inside the map's `<` file rather than the data JSON.

9. Card Sets, Decks & Player Data

9.1 Card data JSON

Each set lives in `<` and is a flat array of card objects.

```
{
  "id":      "base1-1",
  "name":    "Alakazam",
  "supertype": "Pokémon",           // "Pokémon" | "Trainer" | "Energy"
  "subtypes": ["Stage 2"],          // "Basic", "Stage 1", "Stage 2", "Baby",
                                   // "Special" (energies), "ex"
  "hp":      "80",                  // string, only on Pokémon
  "types":   ["Psychic"],           // energy type(s) for Pokémon
  "evolvesFrom": "Kadabra",
  "attacks": [ ... ],
  "weaknesses": [ { "type": "Psychic", "value": "x2" } ],
  "retreatCost": ["Colorless", "Colorless", "Colorless"],
  "convertedRetreatCost": 3,
  "rarity":   "Rare Holo"
}
```

Card image path: `<`

9.2 Player decks

Player deck files live in < (and starter seeds in <). The format is a simple array:

```
[
  { "count": 15, "id": "base1-101" },
  { "count": 4,  "id": "base1-43" },
  { "count": 2,  "id": "base1-27" }
]
```

Field	Type	Notes
<	String	Lowercase < id. Must exist in the matching <.
<	int	Copies of the card in the deck. No engine-enforced cap; deck-builder UI enforces TCG standard limits.

The player's **currently-selected deck name** is the < field of <.

9.3 Player owned cards

< tracks how many copies of each card the player owns:

```
{
  "owned_cards": [
    { "card_id": "base1-1", "owned": 0 },
    { "card_id": "base1-2", "owned": 5 }
  ]
}
```

9.4 Opponent decks

Opponent deck files use the same format as player decks and live at <. The opponent's < field (a string like <) is mapped to the filename.

10. Pack Pricing

File: < **Reader:** <

```
[
  { "pack": "base1", "cost": "100" },
  { "pack": "base5", "cost": "200" },
  { "pack": "neo4", "cost": "550" }
]
```

Field	Notes
<	Set id (matches <)
<	Cash cost, stored as a string but parsed to int

Pack art codes (for gifts and rewards) follow <, e.g. <, <, <. Their images live at <.

11. Coin Shop Inventory

File: < **Reader:** < **Currency:** Munchlax coins (Silver / Gold / Blue) won at the juice vendor.

```
{
  "coins": [
    { "filename": "Arceus Silver.png",      "cost": 300 },
    { "filename": "Gardevoir Silver 2.png", "cost": 300 }
  ]
}
```

Field	Notes
<	Coin image filename including < (lives in <)
<	Coin cost — integer, not string

12. Save / Progress Data Shape

Persisted at < and accessed only via the < autoload (never bypass it — see [\[\[project_gamestate_save_invariant\]\]](#)).

Key fields you'll see referenced in JSON conditions and gift hooks:

Progress key	Type	Set by
<	int	Time cycle / sleep flow
<	< < <	Time cycle
<	int	<, battle rewards
<	< of filenames	<
<	< of lowercase < keys	<
<	<	Gift <
<	< of set ids	Gift <
<	int	< — gates god-pack unlock (>20) and guarantee (every 100). See [[project_pack_opening_counter]].
<	<	Match outro
<	<	First interaction
<	< of NPC names	<
<	< < < <	Shopkeeper state machine
<	bool	Day-1 starter purchase
<	bool	Day-2 free pack drop

Custom < / < conditions read whatever key you point at in this dict, so feel free to invent new boolean flags as long as something elsewhere is setting them.

13. Interactables (Signs, Bed, TV)

Overworld "objects" the player interacts with by pressing **Space** while standing inside the object's zone.

Setup: each map scene has an < **Area2D** node whose children are named < zones (rectangles). The shared script < is attached to that node.

Behaviour is chosen by the shape's NAME:

Shape name	Behaviour
<	Sleep flow — see §13.2
<	Cycling news headlines — see §13.3
anything else	Plain flavour text from < — see §13.1

13.1 Sign flavour text

File: < **Reader:** <

Keyed by **scene name** (the < filename without extension), then by the **shape name**:

```
{
  "Celeste_Harbour": {
    "Sign_Shop_Bakery": "Fresh hot bread from here!",
    "Sign_Beach_Right_Big": "< Magikarp Pond | Pikachu Pond >"
  },
  "Verdant_Forest": { "Sign_North": "Verdant Event Hall ahead." }
}
```

A shape with no matching entry shows a generic fallback line. < / < shapes are handled in code and need no entry.

13.2 Bed

Time of day	Result
< / <	Message: "...it's far too early to go to sleep now!"
<	Yes/No prompt "Would you like to go to sleep now?"

Choosing **Yes** at night: fades to black over < (3.0s), remembers the player's position, advances < +1 and < (via <), then reloads the scene fresh.

13.3 TV news

The TV picks the **highest progression stage** whose condition is currently met, then cycles through that stage's headlines on each press. Conditions are checked top-down in <:

Stage	Condition	Theme
5	<	Verdant Forest open
4	<	SS Anne arrived
3	< or < in <	New sets in store
2	<	Card mart selling packs
1	(default)	Game start

Headline text lives in the < constant in <.

13.4 Adding a new interactable

1. Add a named < under the scene's < node.
2. **Plain text:** add an entry under that scene's key in <.
3. **Custom behaviour:** add a < for the shape name in <.

14. Quick-Reference Cheat Sheet

I want to...	Edit this
Make an opponent require a specific deck	Add < to their entry (§1)
Give a battle special rules (damage boosts, no retreat, forced flips...)	Add < to the opponent entry (§2)
Hide an opponent until a prerequisite is beaten	Add < (§3)
Change how an NPC moves around	Set < plus the matching extra fields (§4)
Add a new shop NPC	< plus the correct < (§5)
Give the player a card / costume / cash	< + < on the NPC (§6)
Make an NPC react to what costume the player wears	< + < (§6.4)
Reward an opponent battle with cash + cards	< + < (§7)
Add a new map-day/time variant	Create < (§8)
Add a new card to a set	Append to < (§9.1)
Change a pack's price	Edit < (§10)
Add a coin to the Coin Shop	Append to < (§11)
Change a sign's flavour text	Edit < (§13.1)
Add a new sign / object interaction	New < under < (§13.4)

Last updated: 2026-06-12